

# 实验报告

## 系统开发工具基础（第三周）

姓名 甘如嫣

学号 25090032014

日期 2026 年 9 月 9 日

# 目录

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>GitHub 仓库与提交记录</b>                        | <b>2</b>  |
| 1.1      | 仓库链接 . . . . .                               | 2         |
| 1.2      | 提交记录 . . . . .                               | 2         |
| <b>2</b> | <b>课上实验检查题</b>                               | <b>3</b>  |
| 2.1      | 题目 9 从源码构建并在干净环境安装 Wheel . . . . .           | 3         |
| 2.2      | 题目 10 让编程智能体进入可验证的修复循环 . . . . .             | 7         |
| 2.3      | 题目 11 把「无法处理」的协作材料改成可执行信息 . . . . .          | 11        |
| 2.4      | 题目 12 修复一个可复现的线性回归训练循环 . . . . .             | 12        |
| <b>3</b> | <b>课后练习</b>                                  | <b>14</b> |
| 3.1      | 实例 1 printenv 对比虚拟环境前后差异 . . . . .           | 14        |
| 3.2      | 实例 2 创建 Python 包并生成 lockfile . . . . .       | 15        |
| 3.3      | 实例 3 编写 Python 应用 Dockerfile . . . . .       | 17        |
| 3.4      | 实例 4 docker-compose 运行应用与 Redis 缓存 . . . . . | 18        |
| 3.5      | 实例 5 三种方式实现两数之和 . . . . .                    | 19        |
| 3.6      | 实例 6 用 AI 生成命令处理 Markdown . . . . .          | 20        |
| 3.7      | 实例 7 氛围编程一个小应用 . . . . .                     | 21        |
| 3.8      | 实例 8 原地修改与输出新文件对比 . . . . .                  | 22        |
| 3.9      | 实例 9 YOLO 模式风险与隔离方案 . . . . .                | 23        |
| 3.10     | 实例 10 查找.py 文件并遍历脚本 . . . . .                | 24        |
| 3.11     | 实例 11 Markdown 无序列表文本流水线 . . . . .           | 25        |
| <b>4</b> | <b>解题感悟</b>                                  | <b>26</b> |

# 1 GitHub 仓库与提交记录

## 1.1 仓库链接

仓库地址：<https://github.com/ganruyan/system-basics-tools>

## 1.2 提交记录

Files

main

Go to file

exercises

q01

q02

q03

q04

report

week2

week3

figs

q09

q09图片

q10

q10图片

q11

q11图片

q12

q12图片

实例1

实例10

实例11

实例2

实例3

实例4

实例5

实例6

实例7

实例8

实例9

| Name         | Last commit message               | Last commit date |
|--------------|-----------------------------------|------------------|
| ..           |                                   |                  |
| figs         | 第三周实验报告 (LaTeX 源码、PDF 与截图)        | 5 minutes ago    |
| q09          | 第9题 从源码构建并在干净环境安装 wheel           | 1 hour ago       |
| q09图片        | 第9题 从源码构建并在干净环境安装 wheel           | 1 hour ago       |
| q10          | 第10题 让编程智能体进入可验证的修复循环             | 1 hour ago       |
| q10图片        | 第10题 让编程智能体进入可验证的修复循环             | 1 hour ago       |
| q11          | 第11题 把协作材料改成可执行信息                 | 1 hour ago       |
| q11图片        | 第11题 把协作材料改成可执行信息                 | 1 hour ago       |
| q12          | 第12题 修复 PyTorch 线性回归训练循环          | 1 hour ago       |
| q12图片        | 第12题 修复 PyTorch 线性回归训练循环          | 1 hour ago       |
| 实例1          | 实例1 printenv 对比虚拟环境前后差异           | 1 hour ago       |
| 实例10         | 实例10 查找 .py 文件并遍历脚本               | 1 hour ago       |
| 实例11         | 实例11 Markdown 无序列表文本流水线           | 1 hour ago       |
| 实例2          | 实例2 创建 Python 包并生成 lockfile       | 1 hour ago       |
| 实例3          | 实例3 编写 Python 应用 Dockerfile       | 1 hour ago       |
| 实例4          | 实例4 docker-compose 运行应用与 Redis 缓存 | 1 hour ago       |
| 实例5          | 实例5 三种方式实现两数之和                    | 1 hour ago       |
| 实例6          | 实例6 用 AI 生成 shell 命令处理 Markdown   | 1 hour ago       |
| 实例7          | 实例7 氛围编程一个小应用                     | 1 hour ago       |
| 实例8          | 实例8 原地修改与输出断文件对比                  | 1 hour ago       |
| 实例9          | 实例9 YOLO 模式风险与隔离方案                | 1 hour ago       |
| 实验报告_第3周.pdf | 第三周实验报告 (LaTeX 源码、PDF 与截图)        | 5 minutes ago    |
| 实验报告_第3周.tex | 第三周实验报告 (LaTeX 源码、PDF 与截图)        | 5 minutes ago    |
| 解题感悟.txt     | 补充解题感悟                            | 1 hour ago       |

图 1: GitHub 提交记录

## 2 课上实验检查题

### 2.1 题目 9 从源码构建并在干净环境安装 Wheel

**主题:** Packaging and Shipping Code **建议用时:** 12—15 分钟

**题目原文:** 在 q09 中按给定内容创建一个最小 Python 命令行包，并验证 wheel 安装。

**给定内容 (src/greetlab/cli.py):**

```
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
```

**给定内容 (pyproject.toml):**

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-学号"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"
```

**题目要求:**

- 创建 src/greetlab/\_\_init\_\_.py、给定的 cli.py 和 pyproject.toml，并把「学号」替换为自己的学号。
- 在已安装 build 工具的课程环境中运行 `python -m build`，生成 wheel。
- 新建一个干净虚拟环境，只从生成的 wheel 安装。
- 切换到 q09 之外运行 `sdt-greet --name < 学号 >`，确认输出正确。

**运行结果:**

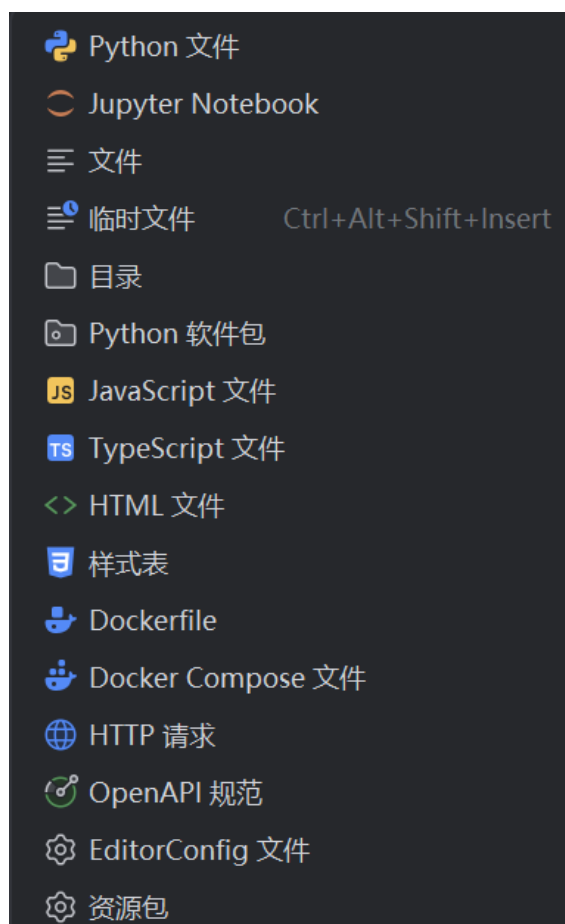


图 2: 创建包文件: pyproject.toml 与 cli.py

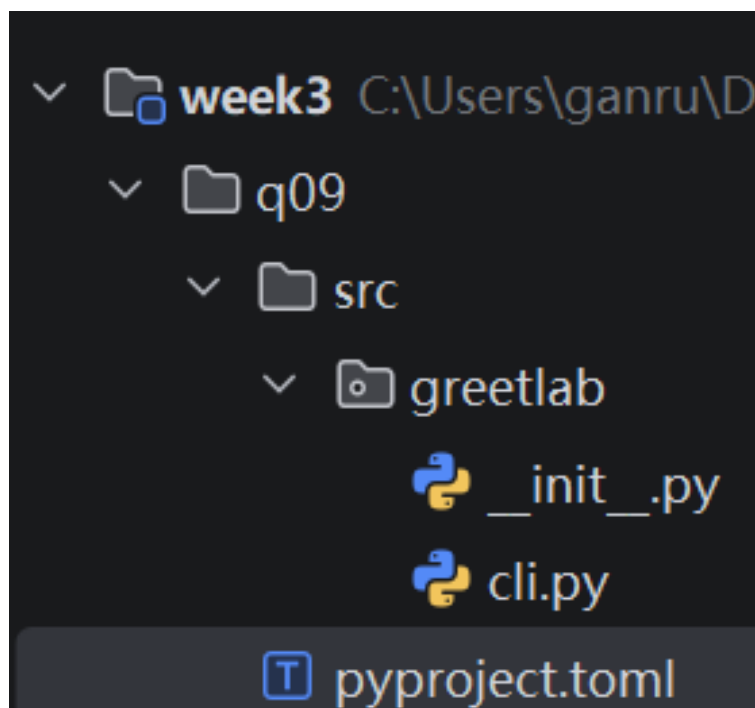


图 3: greetlab 包目录结构

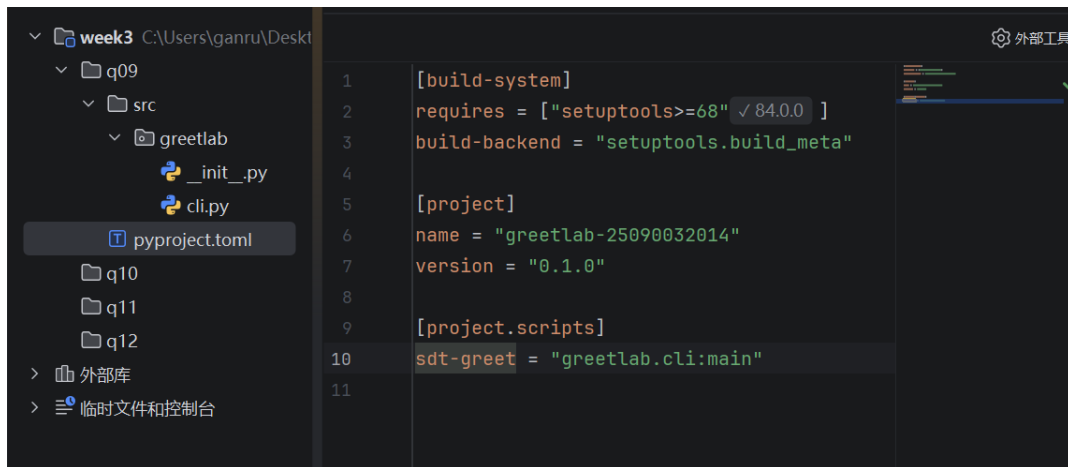


图 4: python -m build 成功生成 wheel

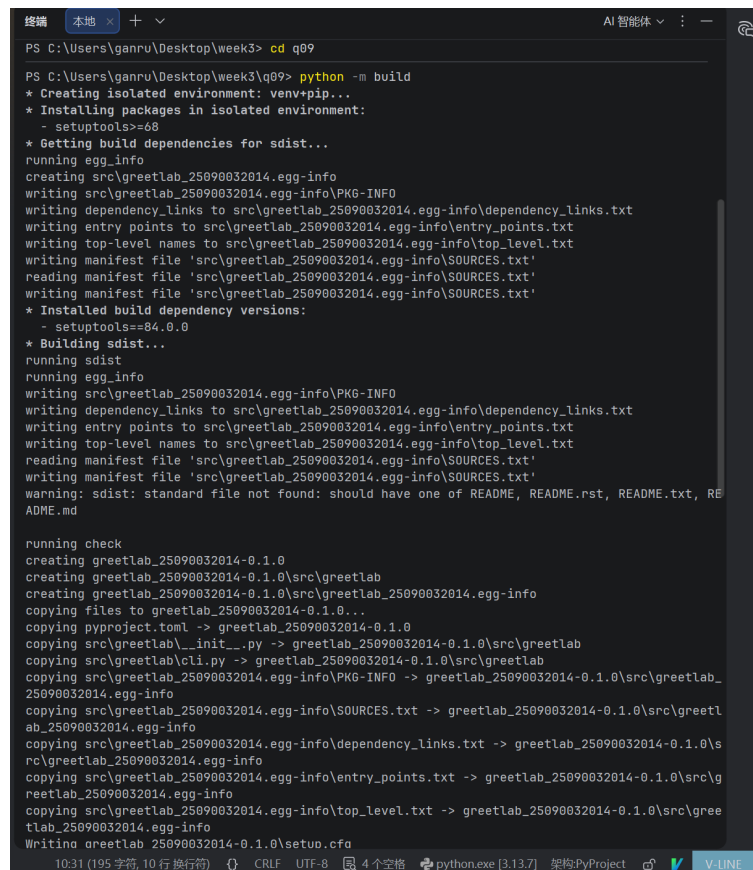


图 5: 干净虚拟环境中 pip install 安装 wheel

```
PS C:\Users\ganru\Desktop\week3\q09> python -m venv venv_test
PS C:\Users\ganru\Desktop\week3\q09>
PS C:\Users\ganru\Desktop\week3\q09>
PS C:\Users\ganru\Desktop\week3\q09> venv_test\Scripts\Activate.ps1
(venv_test) PS C:\Users\ganru\Desktop\week3\q09> venv_test\Scripts\Activate.ps1
(venv_test) PS C:\Users\ganru\Desktop\week3\q09> pip install dist\greetlab_25090032014-0.1.0-py3-none-any.whl
Processing c:\users\ganru\desktop\week3\q09\dist\greetlab_25090032014-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25090032014
Successfully installed greetlab-25090032014-0.1.0

[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv_test) PS C:\Users\ganru\Desktop\week3\q09> cd ..
(venv_test) PS C:\Users\ganru\Desktop\week3> sdt-greet --name 25090032014
Hello, 25090032014!
```

图 6: 运行 sdt-greet --name 25090032014, 输出正确





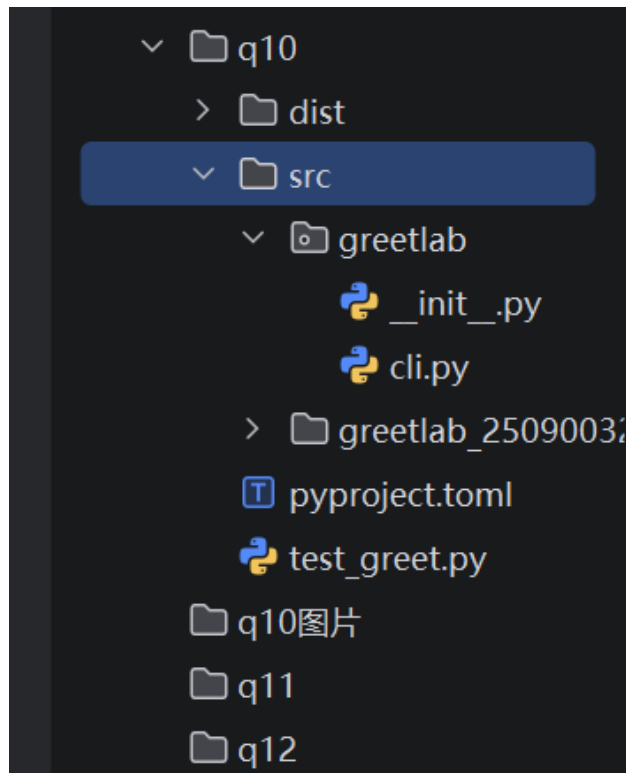


图 8: 向编程智能体说明目标与约束

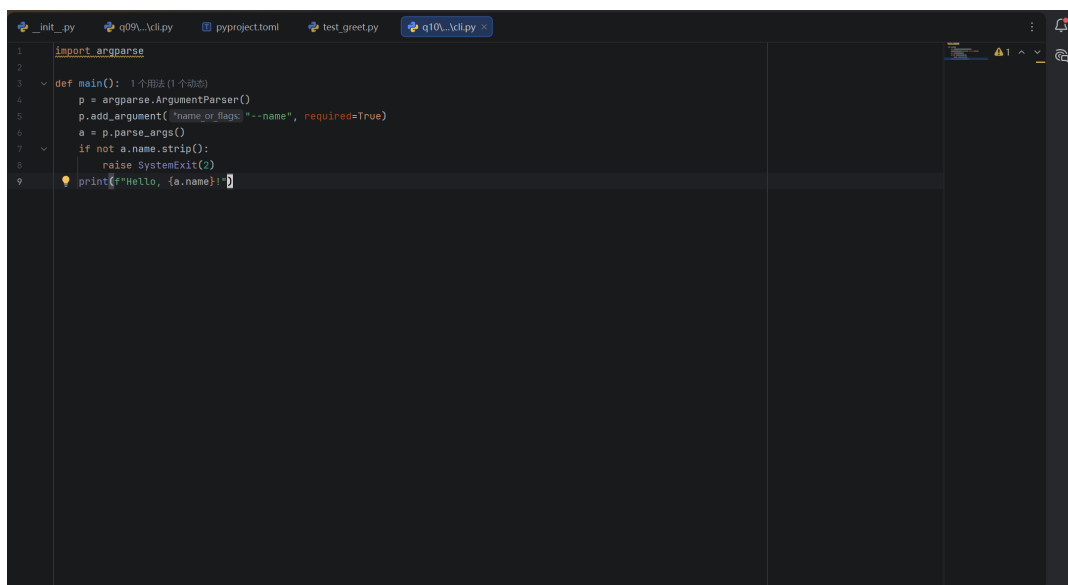
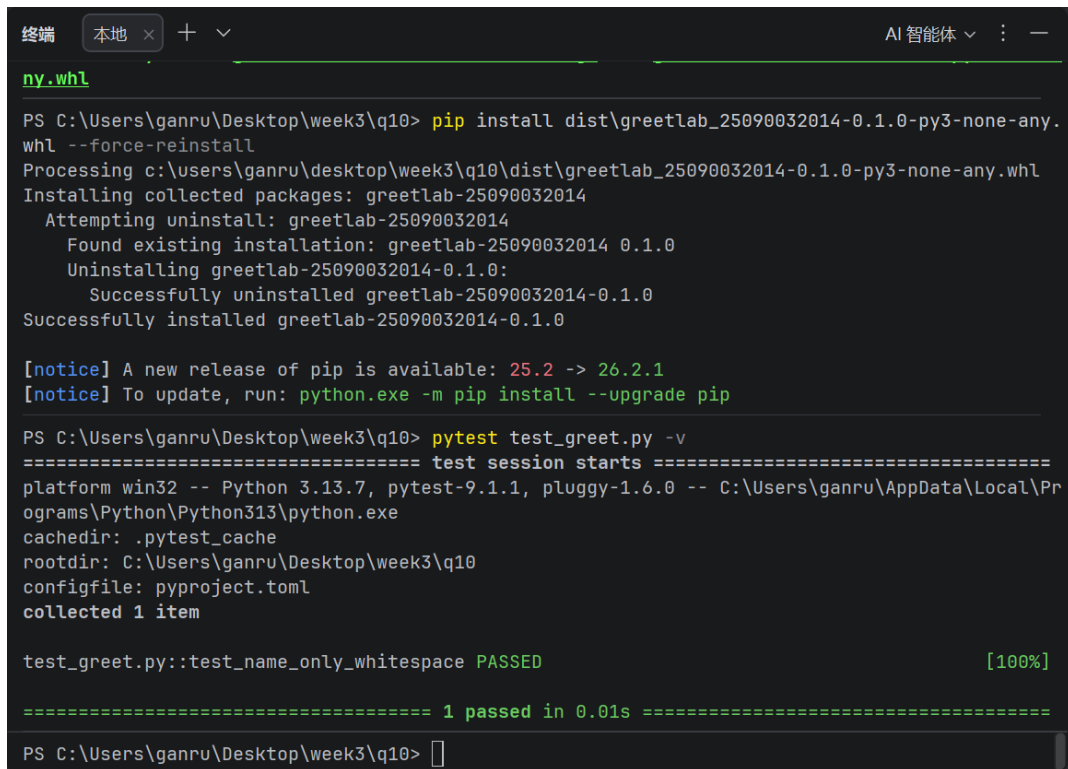


图 9: 人工检查 diff，确认仅改动校验逻辑



```
终端 本地 x + v AI 智能体 v : -
ny.whl

PS C:\Users\ganru\Desktop\week3\q10> pip install dist\greetlab_25090032014-0.1.0-py3-none-any.whl --force-reinstall
Processing c:\users\ganru\desktop\week3\q10\dist\greetlab_25090032014-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25090032014
  Attempting uninstall: greetlab-25090032014
    Found existing installation: greetlab-25090032014 0.1.0
    Uninstalling greetlab-25090032014-0.1.0:
      Successfully uninstalled greetlab-25090032014-0.1.0
    Successfully installed greetlab-25090032014-0.1.0

[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

PS C:\Users\ganru\Desktop\week3\q10> pytest test_greet.py -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\ganru\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ganru\Desktop\week3\q10
configfile: pyproject.toml
collected 1 item

test_greet.py::test_name_only_whitespace PASSED [100%]

===== 1 passed in 0.01s =====

PS C:\Users\ganru\Desktop\week3\q10>
```

图 10: 智能体修改后的 cli.py



图 11: ai\_log.md 记录核心提示、改动与验证

```
PS C:\Users\ganru\Desktop\week3\q10> pip install dist\greetlab_25090032014-0.1.0-py3-none-any.whl --force-reinstall
Processing c:\Users\ganru\Desktop\week3\q10\dist\greetlab_25090032014-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25090032014
  Attempting uninstall: greetlab-25090032014
    Found existing installation: greetlab-25090032014 0.1.0
    Uninstalling greetlab-25090032014-0.1.0:
      Successfully uninstalled greetlab-25090032014-0.1.0
  Successfully installed greetlab-25090032014-0.1.0

[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\ganru\Desktop\week3\q10> pytest test_greet.py -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\ganru\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ganru\Desktop\week3\q10
configfile: pyproject.toml
collected 1 item

test_greet.py::test_name_only_whitespace PASSED [100%]

===== 1 passed in 0.01s =====
PS C:\Users\ganru\Desktop\week3\q10>
```

图 12: 再次运行 pytest, 1 passed

## 2.3 题目 11 把「无法处理」的协作材料改成可执行信息

**主题：**不止于代码 **建议用时：**12—15 分钟

**题目原文：**围绕「空姓名仍输出问候语」这一缺陷，改写下面三段质量较差的协作材料。

**给定内容：**

- Issue: Windows 上运行不了，尽快修复。
- 提交信息: fix bug
- 评审意见: 这里写得不好，重写。
- 已知事实: 运行 `sdt-greet --name " "` 时仍输出 `Hello, !`，并以 `0` 退出。

**题目要求：**

- 在 `communication.md` 中重写 Issue，包含环境、复现命令、期望结果和实际结果；未知信息写「待确认」。
- 重写提交信息：标题使用祈使语气，正文说明问题和解决方案。
- 重写评审意见：指出具体行为、风险和建议动作，并标注 Blocking、Suggestion 或 Nit。
- 全文控制在 400 字以内，保持专业、简洁。

**运行结果：**



图 13: `communication.md` 改写后的内容

## 2.4 题目 12 修复一个可复现的线性回归训练循环

**主题：**Python 与 PyTorch **建议用时：**12—15 分钟

**题目原文：**将下方代码保存为 q12/train.py，补全 TODO 并完成一次 PyTorch CPU 训练。

**给定内容：**

```
import torch
from torch import nn

torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

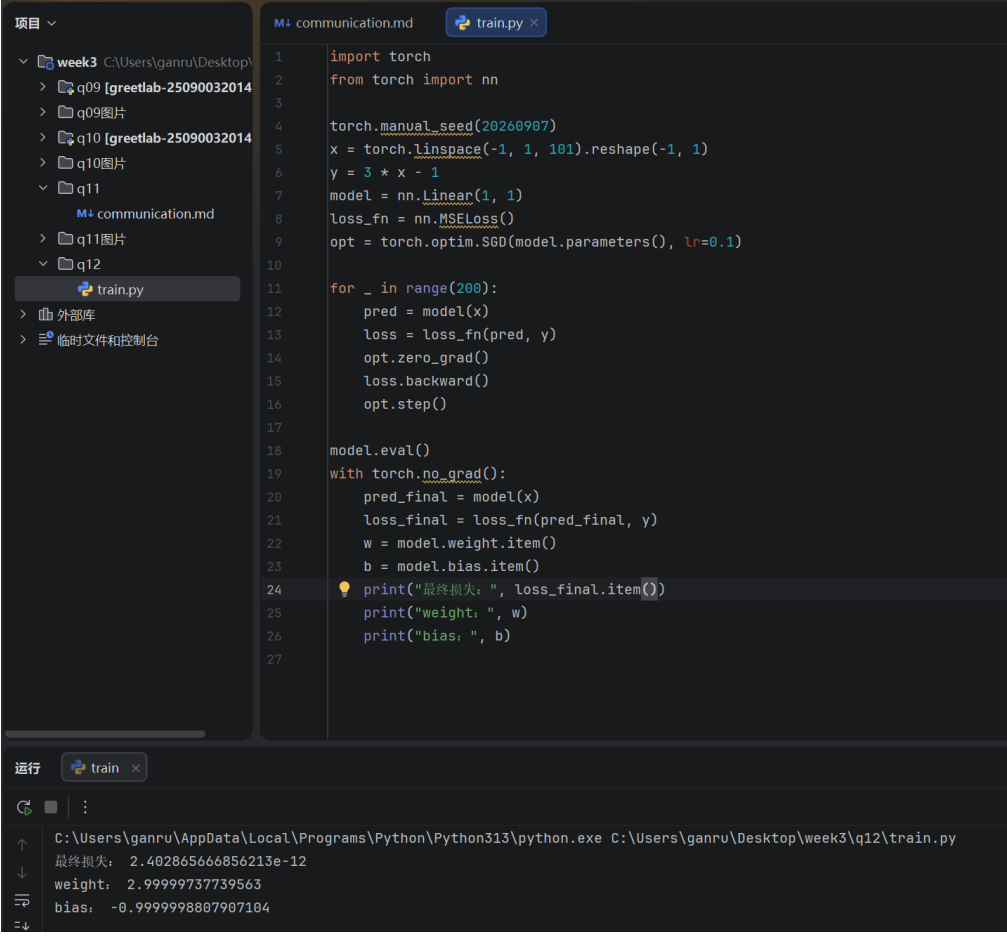
for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    # TODO: 清空梯度、反向传播、更新参数

    # TODO: 进入评估模式并在 no_grad 中打印最终损失、weight 和 bias
```

**题目要求：**

- 补全训练循环，正确调用 `zero_grad`、`backward` 和 `step`。
- 训练后调用 `model.eval()`，并在 `torch.no_grad()` 中计算最终损失。
- 打印最终损失、`weight` 和 `bias`；最终损失应小于 0.001。
- 不得直接把 `weight` 和 `bias` 赋值为 3 和 -1。

**运行结果：**



```
1 import torch
2 from torch import nn
3
4 torch.manual_seed(20260907)
5 x = torch.linspace(-1, 1, 101).reshape(-1, 1)
6 y = 3 * x - 1
7 model = nn.Linear(1, 1)
8 loss_fn = nn.MSELoss()
9 opt = torch.optim.SGD(model.parameters(), lr=0.1)
10
11 for _ in range(200):
12     pred = model(x)
13     loss = loss_fn(pred, y)
14     opt.zero_grad()
15     loss.backward()
16     opt.step()
17
18 model.eval()
19 with torch.no_grad():
20     pred_final = model(x)
21     loss_final = loss_fn(pred_final, y)
22     w = model.weight.item()
23     b = model.bias.item()
24     print("最终损失: ", loss_final.item())
25     print("weight: ", w)
26     print("bias: ", b)
27
```

运行 train

C:\Users\ganru\AppData\Local\Programs\Python\Python313\python.exe C:\Users\ganru\Desktop\week3\q12\train.py

最终损失: 2.402865666856213e-12  
weight: 2.99999737739563  
bias: -0.9999998807907104

图 14: train.py 补全后运行，输出 loss / weight / bias

### 3 课后练习

本次课后练习共完成 **11 个实例**（实例 1~11），满足「每次课完成 >10 个实例」的要求。下面每个实例先放题目原文，再放运行结果截图。

#### 3.1 实例 1 printenv 对比虚拟环境前后差异

**题目原文：** Save your environment with printenv to a file, create a venv, activate it, printenv to another file and diff before.txt after.txt. What changed in the environment? Why does the shell prefer the venv? (Hint: look at \$PATH before and after activation.) Run which deactivate and reason about what the deactivate bash function is doing.

**运行结果：**

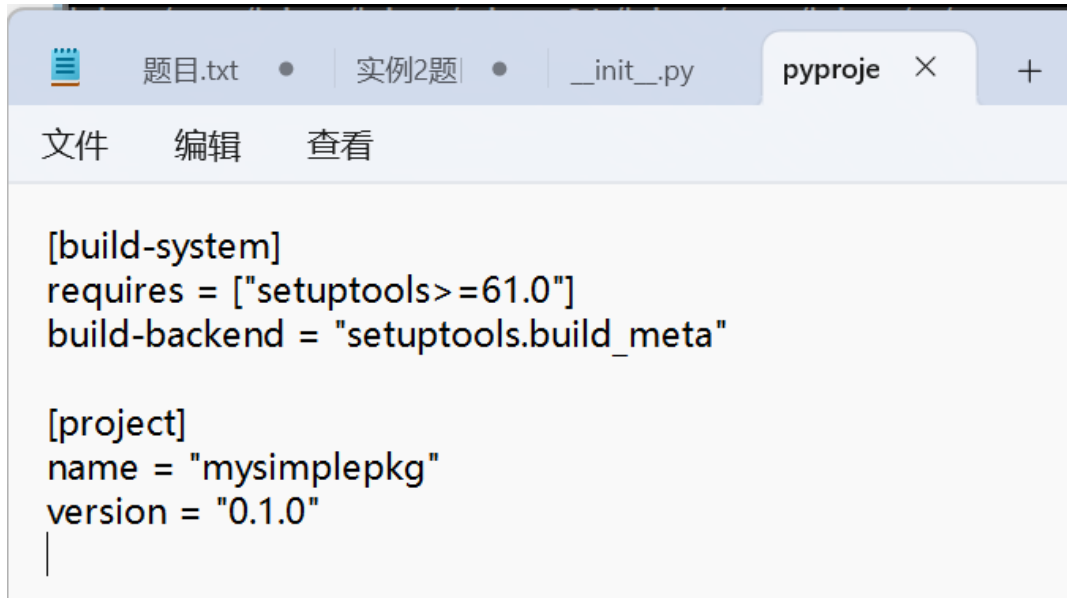
```
ganru@MINGW64 ~
$ source myvenv/Scripts/activate
(myvenv)
ganru@MINGW64 ~
$ printenv > after.txt
(myvenv)
ganru@MINGW64 ~
$ diff before.txt after.txt
35a36
> VIRTUAL_ENV=/c:/Users/ganru/myvenv
59a61
> VIRTUAL_ENV_PROMPT=myvenv
66c68
< PS1=\[\033]0;$TITLEPREFIX:$PWD\007\]\n\[\033[32m]\u@\h \[\033[35m]$MSYSTEM \
\[\033[33m]\w\[\033[36m]`__git_ps1` \[\033[0m]\n$
---
> PS1=(myvenv) \[\033]0;$TITLEPREFIX:$PWD\007\]\n\[\033[32m]\u@\h \[\033[35m]$
MSYSTEM \[\033[33m]\w\[\033[36m]`__git_ps1` \[\033[0m]\n$
71c73
< PATH=/c:/Users/ganru/bin:/mingw64/bin:/usr/local/bin:/usr/bin:/bin:/mingw64/bin
:/usr/bin:/c:/Users/ganru/bin:/d/bin:/c:/windows/system32:/c:/windows/sy
stem32/wbem:/c:/windows/System32/WindowsPowerShell/v1.0:/c:/windows/System32/OpenS
SH:/c:/Program Files (x86)/NVIDIA Corporation/PhysX/Common:/c:/Program Files/NVIDI
A Corporation/NVIDIA app/NvDLISR:/c:/Program Files (x86)/pcsuite:/c:/Program Files
/PUTTY:/c:/Program Files/TortoiseSVN/bin:/d/windows Kits/10/Windows Performance T
oolkit:/cmd:/c:/Program Files/nodejs:/d/Matlab/bin:/c:/Users/ganru/AppData/Local/P
rograms/Python/Python313/Scripts:/c:/Users/ganru/AppData/Local/Programs/Python/py
thon313:/c:/Users/ganru/AppData/Local/Programs/Python/Launcher:/c:/Users/ganru/App
Data/Local/Microsoft/WindowsApps:/c:/Users/ganru/AppData/Local/Programs/Microsoft
 VS Code/bin:/c:/Users/ganru/AppData/Local/GitHubDesktop/bin:/c:/Users/ganru/AppDa
ta/Roaming/npm:/usr/bin/vendor_perl:/usr/bin/core_perl
---
> PATH=/c:/Users/ganru/myvenv/Scripts:/c:/Users/ganru/bin:/mingw64/bin:/usr/local/
bin:/usr/bin:/bin:/mingw64/bin:/usr/bin:/c:/Users/ganru/bin:/d/bin:/c:/windows/sy
stem32:/c:/windows/System32/wbem:/c:/windows/System32/WindowsPowerShell/
v1.0:/c:/windows/System32/OpenSSH:/c:/Program Files (x86)/NVIDIA Corporation/PhysX
/Common:/c:/Program Files/NVIDIA Corporation/NVIDIA app/NvDLISR:/c:/Program Files
(x86)/pcsuite:/c:/Program Files/PUTTY:/c:/Program Files/TortoiseSVN/bin:/d/windows
Kits/10/Windows Performance Toolkit:/cmd:/c:/Program Files/nodejs:/d/Matlab/bin:/
c:/Users/ganru/AppData/Local/Programs/Python/Python313/Scripts:/c:/Users/ganru/App
Data/Local/Programs/Python/Python313:/c:/Users/ganru/AppData/Local/Programs/Pyth
on/Launcher:/c:/Users/ganru/AppData/Local/Microsoft/WindowsApps:/c:/Users/ganru/App
Data/Local/Programs/Microsoft VS Code/bin:/c:/Users/ganru/AppData/Local/GitHubDe
sktop/bin:/c:/Users/ganru/AppData/Roaming/npm:/usr/bin/vendor_perl:/usr/bin/core_
perl
(myvenv)
ganru@MINGW64 ~
$ which deactivate
which: no deactivate in (/c:/Users/ganru/myvenv/Scripts:/c:/Users/ganru/bin:/mingw
64/bin:/usr/local/bin:/usr/bin:/bin:/mingw64/bin:/usr/bin:/c:/Users/ganru/bin:/d/
bin:/c:/windows/system32:/c:/windows/System32/wbem:/c:/windows/System32/
WindowsPowerShell/v1.0:/c:/windows/System32/OpenSSH:/c:/Program Files (x86)/NVIDIA
Corporation/PhysX/Common:/c:/Program Files/NVIDIA Corporation/NVIDIA app/NvDLISR
:/c:/Program Files (x86)/pcsuite:/c:/Program Files/PUTTY:/c:/Program Files/Tortoise
SVN/bin:/d/windows Kits/10/Windows Performance Toolkit:/cmd:/c:/Program Files/nod
ejs:/d/Matlab/bin:/c:/Users/ganru/AppData/Local/Programs/Python/Python313/Scripts
:/c:/Users/ganru/AppData/Local/Programs/Python/Python313:/c:/Users/ganru/AppData/L
ocal/Programs/Python/Launcher:/c:/Users/ganru/AppData/Local/Microsoft/WindowsApps
:/c:/Users/ganru/AppData/Local/Programs/Microsoft VS Code/bin:/c:/Users/ganru/AppD
ata/Local/GitHubDesktop/bin:/c:/Users/ganru/AppData/Roaming/npm:/usr/bin/vendor_p
erl:/usr/bin/core_perl)
```

图 15: printenv 导出环境变量，激活 venv 后 diff 前后差异

## 3.2 实例 2 创建 Python 包并生成 lockfile

**题目原文：** Create a Python package with a pyproject.toml and install it in a virtual environment. Create a lockfile and inspect it.

**运行结果：**



```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "mysimplepkg"
version = "0.1.0"
|
```

图 16: 创建 Python 包 (pyproject.toml)



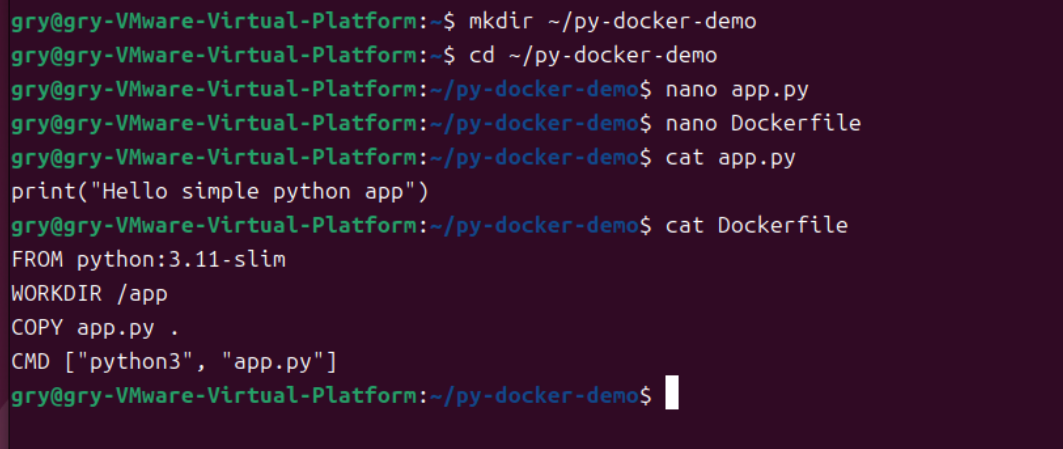
```
ganru@MINGW64 ~/simple_pkg
$ echo "requests" > requirements.in
(venv)
ganru@MINGW64 ~/simple_pkg
$ pip-compile requirements.in
WARNING: --strip-extras is becoming the default in version 8.0.0. To silence this warning, either use --strip-extras to opt into the new default or use --no-strip-extras to retain the existing behavior.
#
# This file is autogenerated by pip-compile with Python 3.13
# by the following command:
#
#   pip-compile --no-index requirements.in
#
certifi==2026.7.22
# via requests
charset-normalizer==3.5.1
# via requests
idna==3.19
# via requests
requests==2.34.2
# via -r requirements.in
urllib3==2.7.0
# via requests
(venv)
ganru@MINGW64 ~/simple_pkg
$ cat requirements.txt
#
# This file is autogenerated by pip-compile with Python 3.13
# by the following command:
#
#   pip-compile --no-index requirements.in
#
certifi==2026.7.22
# via requests
charset-normalizer==3.5.1
# via requests
idna==3.19
# via requests
requests==2.34.2
# via -r requirements.in
urllib3==2.7.0
# via requests
(venv)
ganru@MINGW64 ~/simple_pkg
$
```

图 17: 在虚拟环境安装并生成 lockfile

### 3.3 实例 3 编写 Python 应用 Dockerfile

题目原文：Write a Dockerfile for a simple Python application

运行结果：



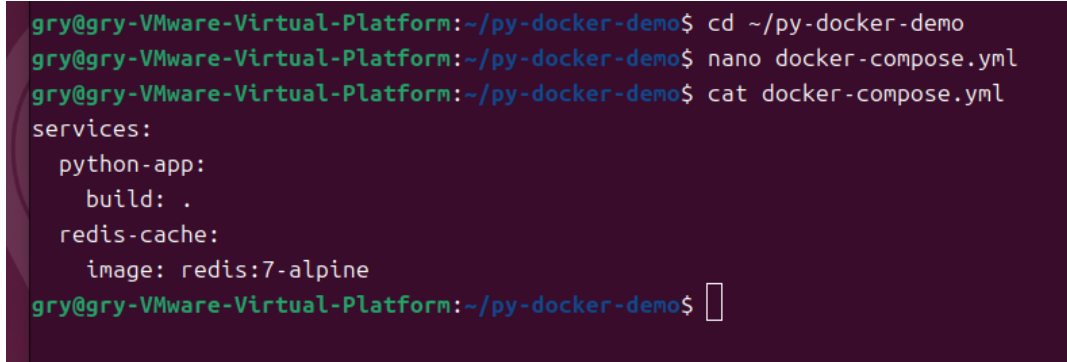
```
gry@gry-VMware-Virtual-Platform:~$ mkdir ~/py-docker-demo
gry@gry-VMware-Virtual-Platform:~$ cd ~/py-docker-demo
gry@gry-VMware-Virtual-Platform:~/py-docker-demo$ nano app.py
gry@gry-VMware-Virtual-Platform:~/py-docker-demo$ nano Dockerfile
gry@gry-VMware-Virtual-Platform:~/py-docker-demo$ cat app.py
print("Hello simple python app")
gry@gry-VMware-Virtual-Platform:~/py-docker-demo$ cat Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
CMD ["python3", "app.py"]
gry@gry-VMware-Virtual-Platform:~/py-docker-demo$
```

图 18: 编写的 Python 应用 Dockerfile

### 3.4 实例 4 docker-compose 运行应用与 Redis 缓存

**题目原文：** Then write a docker-compose.yml that runs your application alongside a Redis cache

**运行结果：**



```
gry@gry-VMware-Virtual-Platform:~/py-docker-demo$ cd ~/py-docker-demo
gry@gry-VMware-Virtual-Platform:~/py-docker-demo$ nano docker-compose.yml
gry@gry-VMware-Virtual-Platform:~/py-docker-demo$ cat docker-compose.yml
services:
  python-app:
    build: .
  redis-cache:
    image: redis:7-alpine
gry@gry-VMware-Virtual-Platform:~/py-docker-demo$
```

图 19: docker-compose.yml (应用 + Redis 缓存)

### 3.5 实例 5 三种方式实现两数之和

**题目原文：**把同一个简单编程任务（选取 LeetCode 两数之和）分别用手动编码、AI 自动补全、AI 聊天对话三种方式实现，对比三者体验差异；可选尝试智能体模式。

**运行结果：**

```
gry@gry-VMware-Virtual-Platform:~$ mkdir ~/two-sum
gry@gry-VMware-Virtual-Platform:~$ cd ~/two-sum
gry@gry-VMware-Virtual-Platform:~/two-sum$ nano two_sum_manual.py
gry@gry-VMware-Virtual-Platform:~/two-sum$ nano two_sum_autocomplete.py
gry@gry-VMware-Virtual-Platform:~/two-sum$ nano two_sum_chat.py
gry@gry-VMware-Virtual-Platform:~/two-sum$ nano compare.md
gry@gry-VMware-Virtual-Platform:~/two-sum$ cat two_sum_manual.py
def two_sum(nums, target):
    for i in range(len(nums)):
        for j in range(i+1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]

print(two_sum([2, 7, 11, 15], 9))
gry@gry-VMware-Virtual-Platform:~/two-sum$ cat compare.md
# 三种编码方式体验对比
任务：LeetCode 两数之和

1. 手动编码 two_sum_manual.py
优点：逻辑完全自己掌控，清楚每一步流程，不存在逻辑陌生问题。
缺点：编写速度慢，边界条件需要自己全部考虑，容易笔误。

2. AI自动补全 two_sum_autocomplete.py
优点：写代码片段时提升速度，框架自己定，AI填充细节。
缺点：只适合小段代码，整体架构仍需要人自己设计，无法独立完成完整算法。

3. AI聊天对话生成 two_sum_chat.py
优点：一次性输出完整可运行代码，速度最快，可以直接给出优化思路（哈希表解法）。
缺点：容易忽略题目隐藏边界，部分代码思路自己不熟悉，需要额外阅读理解。
gry@gry-VMware-Virtual-Platform:~/two-sum$
```

图 20: 手动编码 / AI 补全 / AI 对话三种实现两数之和

### 3.6 实例 6 用 AI 生成命令处理 Markdown

**题目原文：**利用 AI 完成 Markdown 无序列表提取任务。思考 AI 直接修改文件存在什么缺点。让 AI 输出 shell 命令 (grep/sed)，人工手动执行命令完成文本筛选，禁止 AI 直接修改原始 md 文件。

**运行结果：**

```
gry@gry-VMware-Virtual-Platform:~$ mkdir ~/md-demo
gry@gry-VMware-Virtual-Platform:~$ cd ~/md-demo
gry@gry-VMware-Virtual-Platform:~/md-demo$ nano test.md
gry@gry-VMware-Virtual-Platform:~/md-demo$ grep '^-' test.md
- 清单1
- 清单2
- 清单3
gry@gry-VMware-Virtual-Platform:~/md-demo$ nano file_edit_limit.txt
gry@gry-VMware-Virtual-Platform:~/md-demo$
```

图 21: 用 AI 生成 grep/sed 命令，人工执行处理 Markdown 无序列表

### 3.7 实例 7 氛围编程一个小应用

题目原文：从零开始氛围编程一个小应用。不要手动编写一行代码。

运行结果：

```
gry@gry-VMware-Virtual-Platform:~$ mkdir ~/calc-demo
gry@gry-VMware-Virtual-Platform:~$ cd ~/calc-demo
gry@gry-VMware-Virtual-Platform:~/calc-demo$ nano calc.py
gry@gry-VMware-Virtual-Platform:~/calc-demo$ cat calc.py
def calculator():
    print("====简易计算器====")
    try:
        num1 = float(input("请输入第一个数字: "))
        op = input("请输入运算符(+ - * /): ")
        num2 = float(input("请输入第二个数字: "))

        if op == "+":
            res = num1 + num2
        elif op == "-":
            res = num1 - num2
        elif op == "*":
            res = num1 * num2
        elif op == "/":
            if num2 == 0:
                print("错误: 不能除以0! ")
                return
            res = num1 / num2
        else:
            print("运算符非法! ")
            return
        print(f"计算结果: {res}")
    except ValueError:
        print("输入无效, 请输入数字! ")

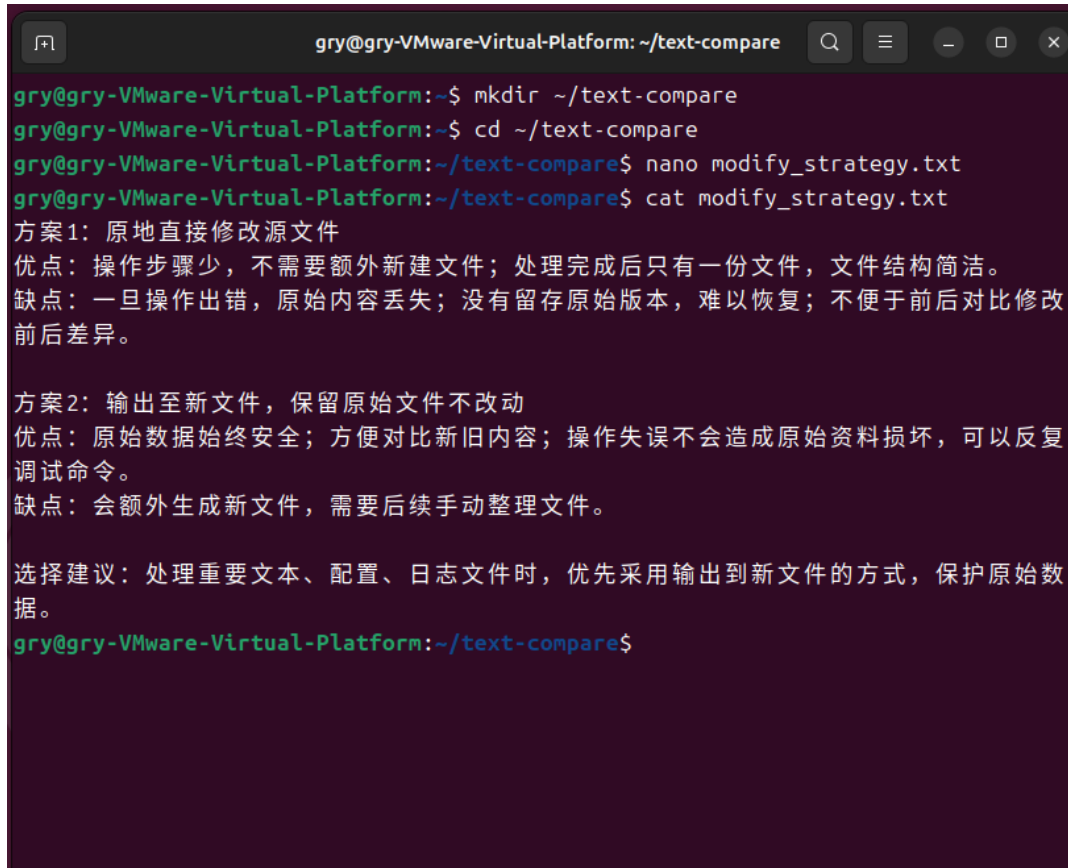
if __name__ == "__main__":
    calculator()
gry@gry-VMware-Virtual-Platform:~/calc-demo$ python3 calc.py
====简易计算器====
请输入第一个数字: 4
请输入运算符(+ - * /): /
请输入第二个数字: 2
计算结果: 2.0
gry@gry-VMware-Virtual-Platform:~/calc-demo$
```

图 22: 氛围编程生成的小应用运行结果

### 3.8 实例 8 原地修改与输出新文件对比

**题目原文：**对比两种文本处理思路：直接原地修改源文件和先输出结果到新文件、不改动原始文件，分别简述优缺点，并说明日常处理重要文本时优先选择哪种方案。

**运行结果：**



```
gry@gry-VMware-Virtual-Platform: ~/text-compare
gry@gry-VMware-Virtual-Platform:~$ mkdir ~/text-compare
gry@gry-VMware-Virtual-Platform:~$ cd ~/text-compare
gry@gry-VMware-Virtual-Platform:~/text-compare$ nano modify_strategy.txt
gry@gry-VMware-Virtual-Platform:~/text-compare$ cat modify_strategy.txt
方案1：原地直接修改源文件
优点：操作步骤少，不需要额外新建文件；处理完成后只有一份文件，文件结构简洁。
缺点：一旦操作出错，原始内容丢失；没有留存原始版本，难以恢复；不便于前后对比修改前后差异。

方案2：输出至新文件，保留原始文件不改动
优点：原始数据始终安全；方便对比新旧内容；操作失误不会造成原始资料损坏，可以反复调试命令。
缺点：会额外生成新文件，需要后续手动整理文件。

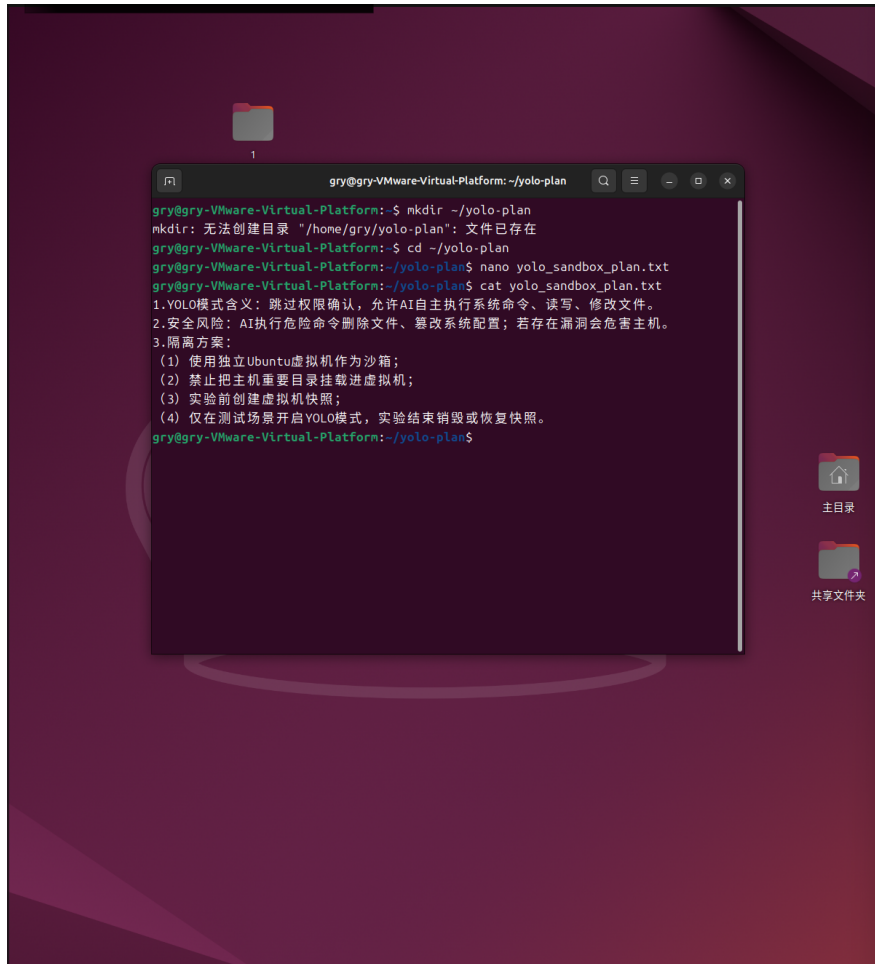
选择建议：处理重要文本、配置、日志文件时，优先采用输出到新文件的方式，保护原始数据。
gry@gry-VMware-Virtual-Platform:~/text-compare$
```

图 23: 原地修改与输出新文件两种思路对比

### 3.9 实例 9 YOLO 模式风险与隔离方案

**题目原文：**调研 AI 智能体 YOLO 自主模式的含义，分析该模式存在的安全风险，写出一份基于 Ubuntu 虚拟机的隔离运行方案，不需要真实搭建部署。

**运行结果：**



```
gry@gry-VMware-Virtual-Platform: ~/yolo-plan
gry@gry-VMware-Virtual-Platform:~$ mkdir ~/yolo-plan
mkdir: 无法创建目录 "/home/gry/yolo-plan": 文件已存在
gry@gry-VMware-Virtual-Platform:~$ cd ~/yolo-plan
gry@gry-VMware-Virtual-Platform:~/yolo-plan$ nano yolo_sandbox_plan.txt
gry@gry-VMware-Virtual-Platform:~/yolo-plan$ cat yolo_sandbox_plan.txt
1.YOLO模式含义：跳过权限确认，允许AI自主执行系统命令、读写、修改文件。
2.安全风险：AI执行危险命令删除文件、篡改系统配置；若存在漏洞会危害主机。
3.隔离方案：
(1) 使用独立Ubuntu虚拟机作为沙箱；
(2) 禁止把主机重要目录挂载进虚拟机；
(3) 实验前创建虚拟机快照；
(4) 仅在测试场景开启YOLO模式，实验结束销毁或恢复快照。
gry@gry-VMware-Virtual-Platform:~/yolo-plan$
```

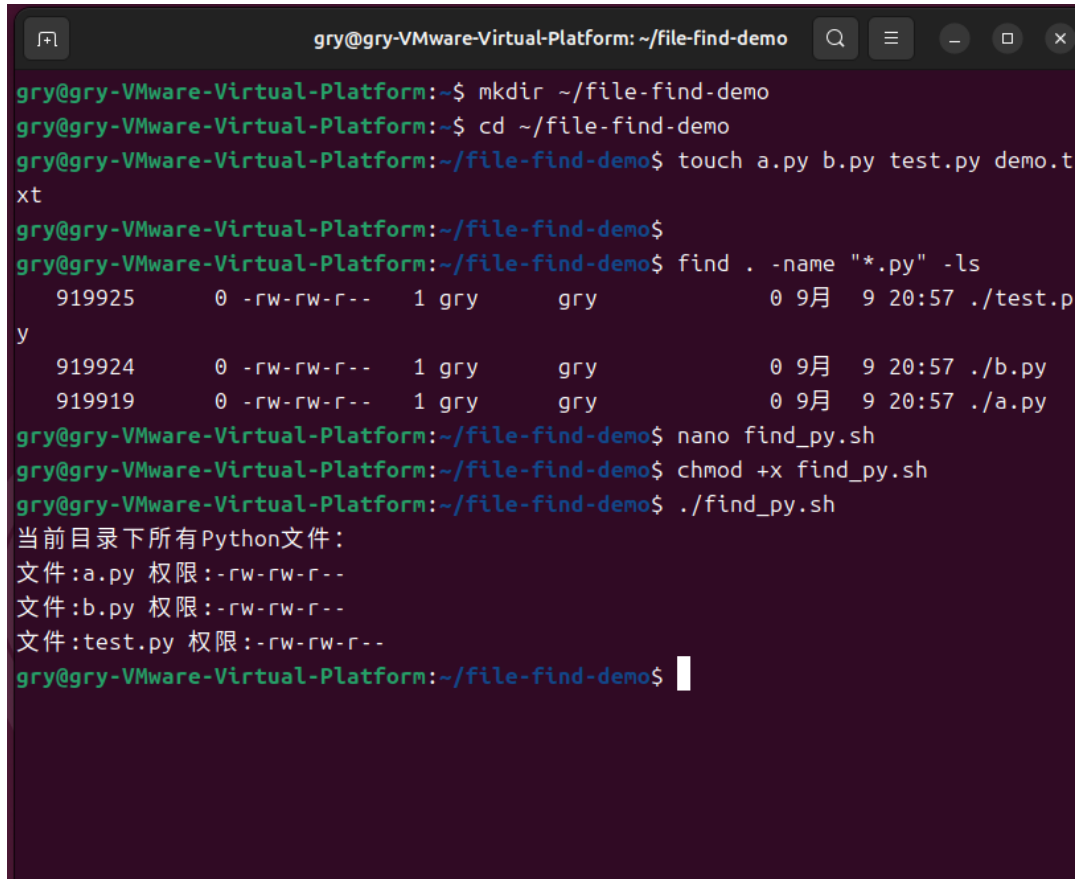
图 24: YOLO 自主模式含义、风险与隔离运行方案



### 3.10 实例 10 查找.py 文件并遍历脚本

**题目原文：**在家目录下，编写一条命令完成：查找当前目录下所有 .py 后缀文件，打印文件名称、文件权限；随后写一段简单 bash 脚本，遍历目录输出所有 Python 文件。

**运行结果：**



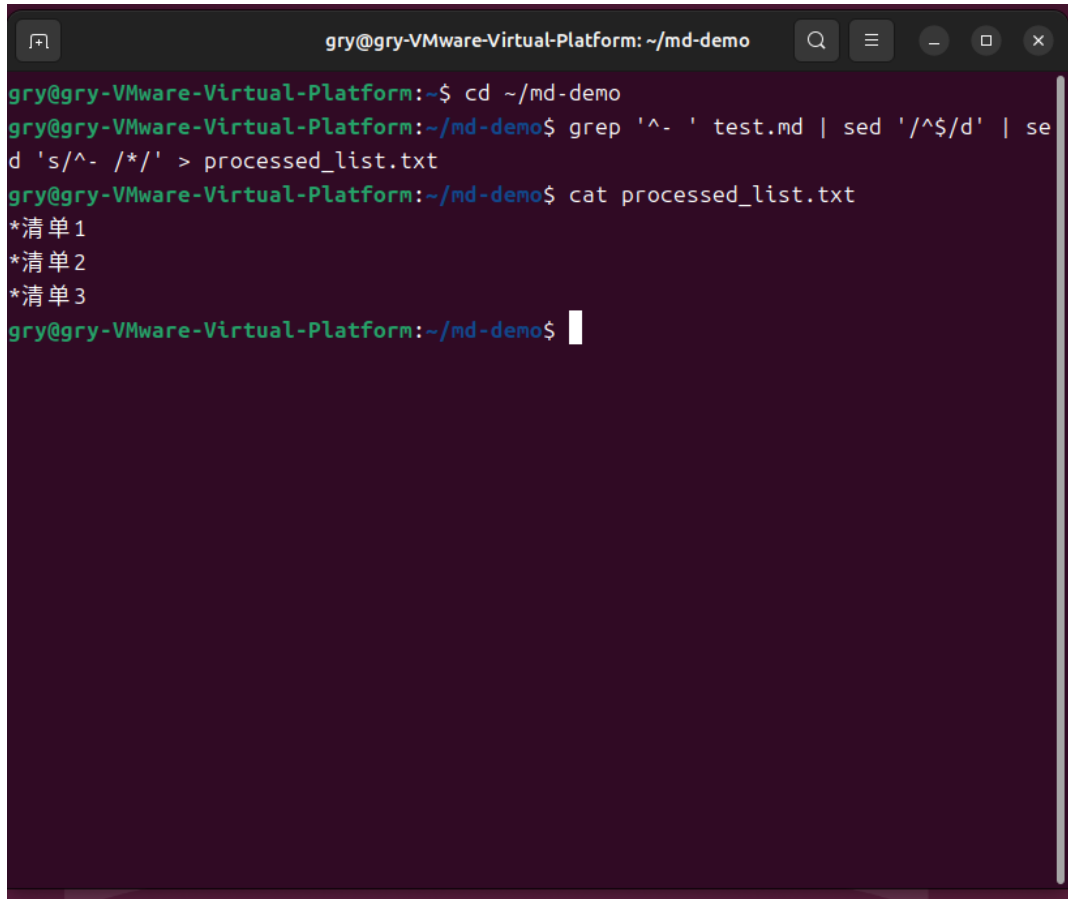
```
gry@gry-VMware-Virtual-Platform: ~/file-find-demo
gry@gry-VMware-Virtual-Platform:~$ mkdir ~/file-find-demo
gry@gry-VMware-Virtual-Platform:~$ cd ~/file-find-demo
gry@gry-VMware-Virtual-Platform:~/file-find-demo$ touch a.py b.py test.py demo.txt
gry@gry-VMware-Virtual-Platform:~/file-find-demo$
gry@gry-VMware-Virtual-Platform:~/file-find-demo$ find . -name "*.py" -ls
  919925      0 -rw-rw-r--   1 gry    gry          0 9月  9 20:57 ./test.py
  919924      0 -rw-rw-r--   1 gry    gry          0 9月  9 20:57 ./b.py
  919919      0 -rw-rw-r--   1 gry    gry          0 9月  9 20:57 ./a.py
gry@gry-VMware-Virtual-Platform:~/file-find-demo$ nano find_py.sh
gry@gry-VMware-Virtual-Platform:~/file-find-demo$ chmod +x find_py.sh
gry@gry-VMware-Virtual-Platform:~/file-find-demo$ ./find_py.sh
当前目录下所有Python文件：
文件:a.py 权限:-rw-rw-r--
文件:b.py 权限:-rw-rw-r--
文件:test.py 权限:-rw-rw-r--
gry@gry-VMware-Virtual-Platform:~/file-find-demo$
```

图 25: 查找.py 文件（名称 + 权限）与遍历脚本输出

### 3.11 实例 11 Markdown 无序列表文本流水线

**题目原文：**对 Markdown 文件进行文本流水线处理：提取无序列表、去除空白行、把开头 - 统一替换成 \*，最终保存到新文件，原始文件保持不变。

**运行结果：**



```
gry@gry-VMware-Virtual-Platform: ~/md-demo
gry@gry-VMware-Virtual-Platform:~$ cd ~/md-demo
gry@gry-VMware-Virtual-Platform:~/md-demo$ grep '^-' test.md | sed '/^$/d' | sed 's/^- /\*/' > processed_list.txt
gry@gry-VMware-Virtual-Platform:~/md-demo$ cat processed_list.txt
*清单1
*清单2
*清单3
gry@gry-VMware-Virtual-Platform:~/md-demo$
```

图 26: Markdown 无序列表文本流水线处理结果

## 4 解题感悟

这次完成实验题目，接触了 Docker 配置、Python 代码和 Linux 命令。一开始我只是照着步骤敲命令，很多地方没有深入思考。最开始只用简单 `grep` 提取文本，做完觉得功能很单薄，后来尝试管道组合多条命令，才感受到 Shell 处理文本的方便。

手动写代码和借助工具生成代码对比下来，我发现工具虽然省事，但生成的内容需要自己检查，不能直接照搬。写配置文件的时候也注意到缩进、格式这类小细节很容易出错。实验过程里印象很深的一点，就是尽量不去改动原始文件，避免误操作弄丢数据。

整体做完收获挺多，不少命令和脚本还不太熟练，之后还要多动手练习，熟悉如何组合工具解决问题。